AI-ASSISTED CODING

ASSIGNMENT-11.1

P.Vishwateja

2303A52389

Batch- 44

TASK-1

Prompt

Generate a Stack class in Python with push, pop, peek, and is_empty methods including proper docstrings

CODE

```python
class Stack:
    """Stack Data Structure (LIFO)"""
    def __init__(self):
        self.items = []

    def push(self, item):
        """Add item to stack"""
        self.items.append(item)

    def pop(self):
        """Remove top item"""
        if not self.is_empty():
            return self.items.pop()
        return "Stack is empty"

    def peek(self):
        """Return top item"""
        if not self.is_empty():
            return self.items[-1]
        return "Stack is empty"

    def is_empty(self):
        return len(self.items) == 0

s = Stack()
s.push(10)
s.push(20)
print(s.peek())
print(s.pop())
print(s.is_empty())
```

OUTPUT

```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-11.1.py
20
20
False
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code>
```

Observation

The Stack follows the **LIFO (Last In First Out)** principle, meaning the last element inserted is removed first.
The push() and pop() operations both take O(1) time, making it efficient.
The peek() method helps to check the top element without removing it.

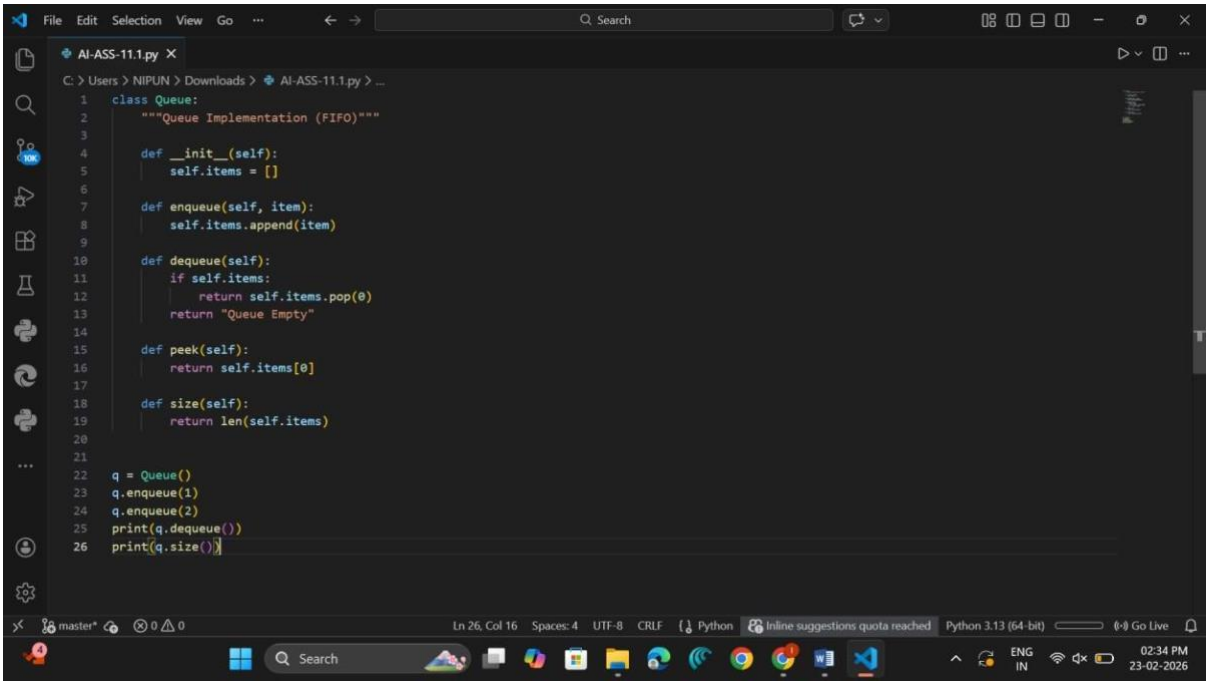
Stacks are commonly used in function calls, undo/redo operations, expression evaluation, and backtracking algorithms.
Since it uses a Python list, memory grows dynamically.

<u>TASK-2</u>

Prompt

Implement a Queue using Python list with enqueue, dequeue, peek, and size methods.

CODE



OUTPUT

```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-11.1.py
1
1
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code>
```

Observation

The Queue follows the **FIFO (First In First Out)** principle, meaning elements are processed in the order they are inserted.
The `enqueue()` operation takes O(1) time, while `dequeue()` using list pop(0) takes O(n) time due to element shifting.
Queues are commonly used in scheduling systems, printer queues, BFS traversal, and service systems where fairness is required.

<u>TASK-3</u>

Prompt

Generate a Singly Linked List with insert and display methods.

CODE

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None
    def insert(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node
    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

ll = LinkedList()
ll.insert(5)
ll.insert(10)
ll.display()
```

OUTPUT

```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-11.1.py
5 -> 10 -> None
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code>
```

Observation

A Linked List stores elements in nodes connected by pointers instead of contiguous memory locations.

It allows dynamic memory allocation and flexible insertion or deletion of elements.
Insertion at the beginning takes O(1), but insertion at the end requires traversal, so it takes O(n).
Linked Lists are useful in implementing stacks, queues, and dynamic memory structures.

TASK-4

Prompt

Create a BST with insert and inorder traversal methods.

CODE

```python
class BST:
    class Node:
        def __init__(self, value):
            self.value = value
            self.left = None
            self.right = None
    def __init__(self):
        self.root = None
    def insert(self, value):
        self.root = self._insert(self.root, value)
    def _insert(self, node, value):
        if not node:
            return self.Node(value)
        if value < node.value:
            node.left = self._insert(node.left, value)
        else:
            node.right = self._insert(node.right, value)
        return node
    def inorder(self, node):
        if node:
            self.inorder(node.left)
            print(node.value, end=" ")
            self.inorder(node.right)

tree = BST()
tree.insert(50)
tree.insert(30)
tree.insert(70)
tree.inorder(tree.root)
```

OUTPUT

```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-11.1.py
30 50 70
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code>
```

Observation

A Binary Search Tree maintains elements in sorted order by placing smaller values on the left and larger values on the right.
Insertion and search operations take O(log n) time on average, but O(n) in worst case if the tree becomes unbalanced.
In-order traversal prints elements in ascending sorted order.
BSTs are widely used in searching, sorting, and database indexing applications.

TASK-5

Prompt

Implement a Hash Table with insert and search methods.

CODE

```python
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self._hash(key)
        self.table[index].append((key, value))

    def search(self, key):
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return "Not Found"

ht = HashTable()
ht.insert("A", 100)
print(ht.search("A"))
```

OUTPUT

```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-11.1.py
100
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code>
```

Observation

A Hash Table stores key-value pairs using a hash function to compute index positions.
It provides average O(1) time complexity for insert, search, and delete operations.
Collisions are handled using chaining, where multiple values are stored in the same bucket.
Hash Tables are widely used in dictionaries, caching systems, and fast lookup applications.

TASK-6

Prompt

Implement a Graph using adjacency list.

CODE

```python
class Graph:
    def __init__(self):
        self.graph = {}

    def add_vertex(self, v):
        self.graph[v] = []

    def add_edge(self, v1, v2):
        self.graph[v1].append(v2)

    def display(self):
        for v in self.graph:
            print(v, "->", self.graph[v])

g = Graph()
g.add_vertex("A")
g.add_vertex("B")
g.add_edge("A", "B")
g.display()
```

OUTPUT

```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-11.1.py
A -> ['B']
B -> []
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code>
```

Observation

A Graph represents relationships between nodes using edges.
In adjacency list representation, each vertex maintains a list of connected vertices, making it memory efficient.
Graphs are used in route planning, social networks, and communication systems.
Traversal algorithms like BFS and DFS are commonly applied to explore graph structures.

<u>TASK-7</u>

Prompt

Implement a Priority Queue using heapq.

CODE

```python
import heapq

class PriorityQueue:
    def __init__(self):
        self.queue = []

    def enqueue(self, priority, item):
        heapq.heappush(self.queue, (priority, item))

    def dequeue(self):
        return heapq.heappop(self.queue)

pq = PriorityQueue()
pq.enqueue(2, "Low")
pq.enqueue(1, "High")
print(pq.dequeue())
```

OUTPUT

```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-11.1.py
(1, 'High')
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code>
```

Observation

A Priority Queue removes elements based on priority rather than insertion order.
It is implemented using a heap structure, where insertion and deletion take O(log n) time.
Elements with higher priority are processed first.
Priority Queues are used in scheduling systems, Dijkstra's algorithm, and task management systems.

<u>TASK-8</u>

Prompt

Implement a Deque using collections.deque.

CODE

```python
from collections import deque

class DequeDS:
    def __init__(self):
        self.dq = deque()

    def insert_front(self, item):
        self.dq.appendleft(item)

    def insert_rear(self, item):
        self.dq.append(item)

    def remove_front(self):
        return self.dq.popleft()

    def remove_rear(self):
        return self.dq.pop()

d = DequeDS()
d.insert_front(1)
d.insert_rear(2)
print(d.remove_front())
```

OUTPUT

```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-11.1.py
1
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code>
```

Observation

Deque (Double Ended Queue) allows insertion and deletion from both front and rear ends.
Using `collections.deque`, operations such as append and pop take O(1) time.
It provides more flexibility compared to a normal queue.
Deque is commonly used in sliding window problems, palindrome checking, and caching mechanisms.

<u>TASK-9</u>

Prompt

Select suitable data structure for cafeteria order queue and implement it.

CODE

```python
class CafeteriaQueue:
    def __init__(self):
        self.orders = []

    def place_order(self, name):
        self.orders.append(name)

    def serve_order(self):
        return self.orders.pop(0)

cq = CafeteriaQueue()
cq.place_order("Nipun")
cq.place_order("Rahul")
print(cq.serve_order())
```

OUTPUT

```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-11.1.py
Nipun
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code>
```

Observation

In the campus system, different data structures are chosen based on functionality.
Queue is suitable for cafeteria orders and attendance tracking because it follows FIFO order.
Hash Table is useful for event registration due to fast search and deletion.
Graph is ideal for bus scheduling since routes and stops form network connections.
Selecting appropriate data structures improves system efficiency and performance.

<u>TASK-10</u>

Prompt

Implement product search using Hash Table.

CODE

```python
class ProductSearch:
    def __init__(self):
        self.products = {}

    def add_product(self, pid, name):
        self.products[pid] = name

    def search_product(self, pid):
        return self.products.get(pid, "Not Found")

ps = ProductSearch()
ps.add_product("P101", "Laptop")
print(ps.search_product("P101"))
```

OUTPUT

```
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code> & C:\Python313\python.exe c:/Users/NIPUN/Downloads/AI-ASS-11.1.py
Laptop
PS C:\Users\NIPUN\AppData\Local\Programs\Microsoft VS Code>
```

Observation

In the e-commerce system, different data structures solve different problems efficiently.
Hash Table is best for product search because it provides O(1) lookup time.
Queue is suitable for order processing since orders are handled in arrival order.
Priority Queue helps track top-selling products based on sales priority.
Using the correct data structure ensures fast performance and scalability of the system.